

pretrain/final-dataset — the corpus for our 20B MoE

For the whole team. 2026-08-07. Status: plan awaiting owner approval. Nothing has been ingested.

Branch `final-dataset` @ `origin/edullm/perf-threaded-verify-and-gatea` (0.9.1, the deployed validator code).

This document is self-contained. It says what the corpus is, which source supplies each part, why each share is what it is, how to measure whether it worked, and how to draw a smaller subset for the baseline model.

1. Summary

We are building a **~1.0-trillion-token pretraining corpus** for an education-focused Mixture-of-Experts model, steered on MMLU, GSM8K, ARC and HellaSwag.

The corpus ships in **two stages**, because no comparable published run used a single flat mixture:

- **Stage 1 — bulk, 900B, 77% web.** Broad, cheap, high-volume. This is where the model learns language.
- **Stage 2 — cooldown, 100B, 32% web.** Concentrated math, code, QA and reasoning traces. **At our scale this is where the benchmark numbers actually move.**

"Web" here means DCLM + FineWeb-Edu + FinePDFs-Edu. Stated once, because the comparisons in §6 are against other labs' own definitions and the term is not standard. **FinePDFs-Edu is counted as web** — it is PDFs harvested from Common Crawl, so the provenance is web even though the format is not HTML. **Excluding it, stage 1 is 70%** (378 + 252 of 900). Stage 2 is 32% under either reading, since its only web source is DCLM.

§9's baseline "70%" is the same convention, not the exclusive one — it is 38 + 26 + 6 including FinePDFs, and it coincidentally equals stage 1's exclusive figure. Excluding FinePDFs the baseline is 64%. **Every percentage in this document includes FinePDFs-Edu.** When comparing to OLMo 2's 95.1% or OLMo 3's 76.1%, note those are *their* definitions of web, which we have not normalized against ours.

Every token is stored as headerless raw `uint32` little-endian under the **dolma2 tokenizer**. No source exceeds **0.90 epochs**, so nothing repeats within the corpus itself.

The same published corpus serves **both** planned models — the 20B flagship and the 7B baseline — by drawing different ratios over the same labelled shards. §9 explains how.

2. The models

	flagship	baseline
total params	20.00B	7.11B
experts	96 (2 shared + top-4 routed)	32 (2 shared + top-4 routed)
experts active per token	6	6
active params	1.876B	1.873B
sparsity	10.66 : 1	3.80 : 1

The two models have the same active parameter count — 0.17% apart. They share `d_model`, layer count, expert width and top-k; only the expert *count* differs. Three consequences the team should hold onto:

1. **Per-token compute is identical.** The baseline is not cheaper to train per token — at any budget, both cost the same. At the plan's 1.25–2.0T that is \$88k–\$140k and 111–178 days on 8×A100, scaled from §11's anchor. The baseline is cheaper in *memory*, not in compute.

⚠ **Corrected 2026-08-07.** This line previously read "At 400B tokens both cost roughly \$41k / 52 days." That is $1.46\times$ off §11's anchor of 1.0T = \$70k / 89 days — the two imply the same \$32.8/hr for 8×A100 but different throughput, so one of them is stale. §11's is the survivor, because it was re-derived after the H100 retraction; the 400B figure predates it. **Take every cost and duration in this document from §11's table and scale linearly.**

1. **What differs is knowledge capacity, not reasoning capacity.** Memorization tracks *total* parameters; reasoning tracks *active* ones. The baseline holds $2.81\times$ less total, so it forgets more and reasons the same.
2. **This is an expert-count ablation at fixed active compute** — the axis Clark et al. (arXiv 2202.01169) swept deliberately. It is a clean experimental design, and §9 is built on it.

One input we need from the team. Shape figures above use `d_model` 2048 / 24 layers / expert intermediate 1366, derived so that 96 experts land on 20.0B total. **If the real `d_model` or layer count differs, active params move and every token and cost figure in this document moves with them.** Confirm the shape and we will re-derive.

3. Stage 1 — bulk, 900B (90% of training), 77% web

source	share	tokens	pool available	epochs	figure grade
DCLM-baseline	42%	378.0B	744.6B	0.51	MEASURED
FineWeb-Edu	28%	252.0B	1,583.1B	0.16	MEASURED
common-pile/stackv2 (code)	10%	90.0B	707.0B	0.13	DERIVED
FinePDFs-Edu	7%	63.0B	70.0B	0.90	CORRECTED
Nemotron-CC-Math-3+	5%	45.0B	134.0B	0.34	MEASURED 
FinePhrase, one partition (synthetic)	4%	36.0B	123.3B	0.29	MEASURED
academic — peS2o + PubMed + arXiv	2%	18.0B	46.6B	0.39	MEASURED
reference — Wikipedia + pre-1929 books	1%	9.0B	26.2B	0.34	MEASURED
StackExchange	1%	9.0B	25.9B	0.35	MEASURED

4. Stage 2 — cooldown, 100B (10% of training), 32% web

source	share	tokens	pool available	epochs
DCLM-baseline	32%	32.0B	744.6B	0.04
common-pile/stackv2 (code)	18%	18.0B	707.0B	0.03
Nemotron-CC-Math-3+	16%	16.0B	134.0B	0.12
AI2 dolma3 midtraining mix (QA-bearing)	14%	14.0B	100.0B	0.14
reasoning traces / worked examples	8%	8.0B	~50B	0.16
Cosmopedia (synthetic)	4%	4.0B	21.7B	0.18
Nemotron Math-Textbooks	3%	3.0B	27.5B	0.11
academic	3%	3.0B	46.6B	0.06
reference	2%	2.0B	26.2B	0.08

Combined, 1,000B: DCLM 41.0% · FineWeb-Edu 25.2% · code 10.8% · FinePDFs 6.3% · math 6.1% · synthetic 4.3% · QA/reference/academic 6.3%.

 **Those buckets close to 100.0% but two of them are not the sum of the rows above.** Stated so nobody re-derives them and finds a gap:

bucket	41.0	what it actually contains
synthetic 4.3%		FinePhrase 3.6% + Cosmopedia 0.4% + Nemotron Math-Textbooks 0.3% — which §4 lists as its own row, not as synthetic
QA/reference/academic 6.3%		dolma3 QA 1.4 + reasoning 0.8 + academic 2.1 + reference 1.1 + StackExchange 0.9 — StackExchange has its own row in §3

Both are defensible groupings; neither is stated. **If you re-cut these buckets, re-derive from the two tables above, not from this line.**

The epoch columns are PER STAGE and are never summed — sum them yourself

Every epochs figure in §3 and §4 divides that stage's draw by the full pool. Seven sources appear in both stages, so their true total exposure is the sum. The "max epoch 0.90" claim survives only because FinePDFs-Edu — the source at 0.90 — appears in stage 1 alone.

source	stage 1	stage 2	total
DCLM-baseline	0.51	0.04	0.55
Nemotron-CC-Math	0.34	0.12	0.46
academic	0.39	0.06	0.45
reference	0.34	0.08	0.42
code (stackv2)	0.13	0.03	0.16
FinePDFs-Edu	0.90	—	0.90 ← still the max
FineWeb-Edu	0.16	—	0.16

So "max epoch 0.90" is correct, and it is the right thing to check — but it is one arithmetic accident away from being wrong. Any reweighting that adds FinePDFs-Edu to the cooldown, or raises Nemotron-CC-Math past $\sim 2.2\times$ its current draw, breaks the under-1-epoch property that this corpus's whole no-repetition claim rests on. Re-derive this table after any share change.

Grade key, because not every number here carries the same weight. MEASURED = counted under dolma2 from exact parquet-footer byte totals $\times$ sampled tokens-per-byte. DERIVED = byte total measured, token count from a measured ratio. CORRECTED = a published figure we checked and found wrong, replaced with our own. CARD = the dataset card's claim, tokenizer unnamed.

✓ As of 2026-08-07 no CARD figure remains in either table. The last one, Nemotron-CC-Math, was measured under dolma2 from exact footers (472,213,218,716 bytes) $\times$ 0.283686 tok/byte sampled over 1,920 random-offset documents at seed 42 = 134.0B — splitting 83.6B (config 3) + 50.4B (config 4plus). The card's 133B was close. Artifact: `_nemotron_cc_math_dolma2_measure.json`.

5. Corpus size versus training budget — they are two different numbers

We can source about 1.0T unique tokens at acceptable quality. That is the corpus.

The training budget should be higher. Every disclosed model in our size class trained more:

model	active / total	tokens trained
JetMoE	2.2B / 8.0B	1.25T ← class floor
DeepSeekMoE-16B	2.8B / 16.4B	2.0T
Moonlight	2.24B / 15.29B	5.7T
Ling-mini-2.0 (11.4:1 — closest to our 10.66:1)	1.4B / 16.0B	20T
OLMoE	1.3B / 6.9B	5.0T

Nobody in this class trained under 1.25T. So:

- **Build ~1.0T unique.**
- **Train 1.25–2.0T**, which is 1.25–2.0 epochs over that corpus.

At 2.0T the average token is worth ~98.4% of a fresh one, so the repetition is close to free. **Treat 2 epochs as a cap to respect rather than a curve to optimize** — the repetition law behind that figure was fit on *dense* models only, and four labs report values of 4.4 / 11.09 / 15.39 / 23.82 for it on the *same* corpus. It does not reproduce, so do not lean on it.

"1× Chinchilla" **does not define a budget here**. Chinchilla optimizes two free variables; our parameter counts are already fixed by memory and serving targets, so the remaining problem is monotone in tokens with no interior optimum. Note also that $\sqrt{(\text{active} \times \text{total})} = 6.13\text{B}$ is *algebraically* the dense Chinchilla-optimal size at that compute — so "the sqrt reading" and "the total reading" are one claim, not two independent ones.

6. Why two stages, and why 77% then 32%

No comparable published run used a flat share. OLMO 2 §2.3 (arXiv 2501.00656, read directly): stage 1 is " $\geq 90\%$ of training FLOPs" and "*mostly web-sourced*"; stage 2 "*up-sample[s] the highest-quality web documents and curated non-web sources*" plus "*synthetic data crafted to patch math capabilities*." OLMO 2 stage 1 is **95.1% web**, Olmo 3 **76.1%**, Olmix fixes web at **75%**; their stage 2 drops to 27.5–52%.

At our scale the cooldown is where the metrics move. At 1B parameters, 4T tokens of 95%-web pretraining leaves MMLU near chance. Mid-training then moves MMLU +17.4 and GSM8K 3.3 → 43.8. That benefit is *largest at our size*: +37.0% at 1B versus +12.3% at 32B. At 7B, across seven different mid-training mixes, MMLU spans 1.3 points while GSM8K spans 27.5.

Web is DCLM-led rather than FineWeb-Edu-led — on direction only. DCLM's own token-matched comparison at 276B/7B favours DCLM on MMLU, but its hyperparameters were tuned on DCLM-baseline, and under FineWeb-Edu's own eval protocol the same paper says the two "*perform quite similarly*." Nemotron-CC and SmolLM2 place FineWeb-Edu on **opposite sides** of DCLM. So we carry both, weighted toward DCLM, and treat the ranking as unsettled. **"Which web" is worth roughly 16 MMLU points at constant share — 4–8× any share effect** — which is why the split matters more than the percentage.

Math is one artifact, not four. Nemotron-CC-Math re-fetches web pages for URLs harvested from FineMath, OpenWebMath and MegaMath, so it nearly *contains* them; adding any of those alongside it would double-count. It also scores best of the four in a token-matched comparison.

Code is 10% in the bulk and 18% in the cooldown. A 64-run sweep at 470M parameters (arXiv 2408.10914) found reasoning peaks near 25% code while **world knowledge is already –3.4% at 25% and –31% at 75%**. For an education model, code belongs *late and concentrated* rather than spread through the bulk. (Caveat: that sweep ran only at 470M, so it does not bracket our active size.)

7. QA and worked examples

These were requested as first-class categories. The measured evidence says they must be **two** categories, because they move our four headline metrics in **opposite directions**. Both rows below are compute-matched microanneals against a web-only control (OLMo 3):

intervention	MMLU	ARC	GSM8K	Minerva	HumanEval
QA in multiple-choice format	+3.2	+2.6	−1.2	−1.6	−1.5
reasoning traces / worked examples	−1.5	+0.1	+8.4	+7.3	+11.6

They partially cancel. A single blended "QA and worked examples" share would be arithmetically wrong. Hence they appear separately in stage 2: **14% QA-bearing midtraining mix** and **8% reasoning traces**. The 14% is the measured saturation point — AI2 allocated 13.9%, and beyond that the curve is flat (−0.1 / +0.7).

Three things to be honest about:

The QA gain is format-matching, not knowledge. The QA data that produced +3.2 MMLU is model-rewritten *multiple-choice* text — format-matched to MMLU and ARC by construction. It does not transfer to StackExchange, which is prose question-and-answer. Our own P1 experiment measured the ceiling directly: 65.24% accuracy under QA-shaped prompts collapses to **3.02%** under chat-style prompts and **0.00%** under short-answer prompts. P1 §2.6 lists format-independent knowledge as *not supported*.

The honest MMLU expectation from adding QA data is +0. The one experiment above the measurement threshold described in §8 — 8B parameters, 1T tokens, MMLU live at 48–53 — gives synthetic QA **+0.2 / −1.1**. The frequently-quoted "+5.6 MMLU" from that same paper is *classifier selection*, not QA synthesis. And the learning-rate anneal alone gave **+2.0 MMLU** in OLMo 2, which is routinely misattributed to the data.

Worked examples buy math capability and cost something else — twice, independently. OLMo 3 at 7B: **+8.4 GSM8K, −1.5 MMLU**. Our own P1 at 370M: Pass@8 rose 13.49 → 18.21, but **PassRatio@8 fell on every scaffolded arm** (5.11 → 4.33/4.52/4.59), and the pedagogically-ordered schedule *lost* to random order. Capability up, reliability or knowledge down, at two very different scales.

8. How to measure whether any of this worked

Multiple-choice scoring is a separately-acquired skill and is random below roughly 400B tokens. OLMES (arXiv 2406.08446), Figure 1 caption, verbatim: "*During early training, there is good signal from CF while MCF is random. Around 400B tokens, the model starts gaining the ability on the MCF format.*"

This is the single most useful fact in the report for anyone running an ablation. It sorts the whole data-mixture literature into uninformative nulls below the threshold and real zeros above it — and it means **any probe run under ~400B tokens produces an MMLU column that measures nothing**.

So: score CF (cloze), or use bits-per-byte. Our own P1 probes ran 300M–10B tokens, far inside the dead zone; they are valid because they reported bits-per-byte. AI2 separately abandoned accuracy metrics at 1B/100B as "*too difficult to show improvement*." Note the relationship **inverts** at the top end — strong models reach 93.7% MCF versus 69.0% CF on ARC-Challenge — so this is scale-dependent, not a blanket rule. OLMES evaluates both and takes the maximum.

Gate on MMLU and GSM8K directly, at fixed intervals, and treat a knowledge regression as a stop condition. Knowledge collapse is the most reproducible harm in this literature: phi-4's heavily repeated synthetic arm cost TriviaQA – 14.8; MoE data reuse took MMLU 32.92 → 24.57; Qwen2's 12T relaxed-quality run showed no improvement over its 7T run. Repetition costs knowledge and spares reasoning — which is exactly our weighted axis — and no loss-based or bits-per-byte gate can see it.

9. Drawing the baseline subset from the same corpus

The 7.11B baseline does **not** need its own corpus. It draws different ratios over the same published, labelled shards. Two intuitive answers are both wrong:

- "Scale tokens down by parameter count." No. Token budgets scale with the parameters that bear the FLOPs, and those are **unchanged** (1.873B versus 1.876B). A $2.81\times$ token cut has no basis.
- "Same mixture, fewer tokens." Incomplete. It answers volume but not composition, and composition is where the capacity reduction actually binds.

The principled basis: memorization tracks *total* parameters, reasoning tracks *active* ones. The baseline loses $2.81\times$ of its total and keeps active intact — so it loses **knowledge capacity** and keeps **reasoning capacity**. The subset should therefore shed knowledge-shaped data and hold or raise reasoning-shaped data.

source	flagship	baseline	why
DCLM-baseline	42%	38%	knowledge-shaped breadth — cut first
FineWeb-Edu	28%	26%	knowledge-shaped
common-pile/stackv2	10%	13%	reasoning-shaped — raise
Nemotron-CC-Math-3+	5%	8%	reasoning-shaped — raise
FinePDFs-Edu	7%	6%	knowledge-shaped
StackExchange	1%	2%	raise slightly
FinePhrase	4%	4%	hold — format diversity
academic	2%	2%	hold
reference	1%	1%	hold — already at floor

Web share goes 77% → 70%. Because the reader selects whole shards by label predicate, **this is a different ratio vector over the same corpus — no re-ingest, no second dataset**. That is the payoff for labelling properly at build time.

Keep the baseline's token budget the same order as the flagship's. If wall-clock is the constraint, cap steps rather than shrinking the pool: a differently-sized pool adds a confound to the very axis — expert count at fixed active compute — that the baseline exists to measure.

10. Contamination — read this before trusting any evaluation number

Seventeen corpora were audited from papers and source code rather than dataset cards. **None earns an unqualified TRUST verdict.** Two findings are directly actionable:

1. **Nemotron-CC-Math's decontamination gate effectively never fired.** An independent 13-gram scan found **11,868 contaminated documents remaining against MATH500 alone — 13.2× more than the entire removal budget the publisher reported** — including verbatim GSM8K *test* items at Jaccard 1.0. This does not change the source choice, because every alternative scores worse, but it makes mitigation mandatory.
2. **The Common Pile ships GSM8K in Flan chain-of-thought format** (`fc-cot-cot_gsm8k` , 6 repeats, $\sim 9\times$ cooldown upweight). AI2 dropped Winogrande *the benchmark* rather than the data, and GSM8K was not in their evaluation suite, so it never triggered the same reflex. **Fix: drop one source, 0.51% of tokens.**

tier	action	cost
1	exclude 9 items at source — carries nearly all the value	<1% of tokens
2	chunked embedding similarity $\geq 0.5 \rightarrow$ LLM judge, on the cooldown + synthetic + question-bank slices	\$500–2,000
3	13-gram matching as an instrument , not a filter — publish the number	\$10–50
4	do not do — aggressive n-gram decontamination, which is MMLU-destroying	—

Three things everyone should know. **A quality classifier is a contamination amplifier** — the DCLM classifier places *all* MMLU and GSM8K probe items above the 99th percentile, and DCLM is our largest source. **13-gram matching scores F1 0.926 on verbatim text but exactly 0 on rephrased text**, so our synthetic slice is structurally un-decontaminatable by n-grams. And **HellaSwag is negative in four independent papers** while also being where contamination persists longest — our least-served and least-instrumented metric from both directions.

11. Build details

Tokenizer: dolma2. No alternative clears a measurable downstream gain: superword tokenization is a null at MoE scale with a code regression, a larger vocabulary is contradicted by the resolved scaling law, and the right-to-left digit variant is a 29-minute abandoned experiment its authors never trained on. Keeping dolma2 has a second benefit — **AI2's pre-tokenized shards are byte-identical to our format** (verified by range-read: no NumPy header, valid token ids from byte 0), so their 5.93T corpus is a byte copy rather than a re-tokenization.

Shards: 25,001,984 tokens each $\rightarrow$ about 40,000 objects. This is `SHARD_TOKENS = 3052  $\times$  8192` in `corpus.py:89` — the value the code actually uses, and the one every figure in `IMPLEMENTATION-PLAN.md` is computed at. An earlier draft of this line said `50,003,968  $\rightarrow$   $\sim 20,000$  objects`; that number appeared in no code and is withdrawn.

Mixture error from whole-shard selection is bounded by $1 \div \text{shards-per-component}$, which at this shard size is **0.007%–0.278%** across the stage-1 sources. (The **0.33%** figure quoted earlier is the same quantity computed at the withdrawn 50M size.) Either way it is far below the level at which it would constrain the design — see `IMPLEMENTATION-PLAN.md` §8.3.

 **The larger shard is still worth choosing deliberately, on a different basis.** Halving the object count halves Gate A, which is `objects  $\times$  8  $\times$  latency`. That trade is settled in `IMPLEMENTATION-PLAN.md` §8.3:

thread the profile checks instead, and then shard size stops being forced by the validator at all. **Do not change SHARD_TOKENS without re-deriving §8's object counts and every wall-clock figure that rests on them.**

Compute, on hardware that is actually provisioned. H100 shapes are **not** currently available in this account; live profiles are T4 / L4 / L40S / A10G / 8×A100.

training budget	8×A100 (today)	8×H100 (if unblocked)
400B tokens	\$28k / 36 days	\$15k / 11 days
1.0T tokens	\$70k / 89 days	\$37k / 28 days
1.25T (the plan's floor)	\$88k / 111 days	\$46k / 35 days
2.0T tokens	\$140k / 178 days	\$73k / 56 days

This table is the single cost anchor for the whole document — §2's per-token-compute point scales from it. All rows are linear in tokens at **\$32.8/hr** for the 8×A100 shape. The 400B row is included only because §2 previously carried a conflicting figure for that budget; see the correction there.

Unblocking H100 access is worth more than any budget decision available to us.

The one genuinely MoE-specific data requirement: micro-batches must not be domain-pure. Load-balancing loss computed over a domain-pure micro-batch forces uniform routing and suppresses expert specialization — worth **0.13–0.18 perplexity and +5–6 GSM8K**, more than a 50% increase in activated FLOPs buys. Our shards are per-source, so every micro-batch is domain-pure by construction, and the relevant setting defaults to per-batch scope in four places in the trainer. **It must be fixed before training starts:** switching at 10% of the run recovers only ~55% of the gap. Note our config has **only 2 shared experts out of 6 active**, which makes routing quality load-bearing rather than incidental.

Everything else about the corpus we choose exactly as we would for a dense model — Qwen3 published that a MoE reaches dense-equivalent performance *on the same data*.

One decision that cannot be undone later: store a per-document quality percentile as a label, at ingest. Every 2026-era corpus reweights on per-document quality and topic labels. Our labels live inside the manifest hash, so they cannot be added afterwards without re-copying the corpus. A wrong budget is a config field; a wrong schema is a version 2.

12. What is still unverified

Stated plainly. Several published figures in this space do not survive checking, so the distinction between what we measured and what we inherited is worth keeping visible.

- **Nemotron-CC-Math's 133B** is a card claim with no tokenizer named. It needs a real count before it enters mixture arithmetic.
- **FinePDFs-Edu's ~70B** is byte-derived rather than directly counted. A widely-circulated 161.07B figure for it is wrong by 2.3× — it implies a byte-per-token ratio that is not physically possible for English prose on this tokenizer.
- **Gate A's validation cost at ~40,000 objects** is an extrapolation from a measured 85 minutes at 10,049 objects. Never tested at scale.

- **MegaMath-Web's post-filter pool size** is unmeasured; its quality score floor admits documents at 0.40 and only 9.4% of document heads carry LaTeX.
- **Dolma 3's adult-content prevalence** is unmeasured and is **blocking** for that source.
- **The largest residual risk: every mixture study cited in this plan was run on a dense model. No mixture ablation on a sparse MoE exists at any scale.** Transfer to our architecture is unverified, and that unknown is larger than any measurement error in this document.

13. Next steps

This document decides **WHAT** to build. Two companions decide **how and when**, and you need both before writing code or launching a job: - docs/IMPLEMENTATION-PLAN.md — build mechanics, and five defects that would silently corrupt or discard the work. None of them fails loudly. - docs/BUILD-DEPENDENCY-GRAPH.md — the execution DAG, the **21.31 h** critical path, and what may run in parallel. - docs/TASKS.md — every **#NN** id, and the crosswalk to graph nodes and Phase 0 items.

1. **Confirm the model shape** (d_model, layers, expert width) so §2's active-parameter figures and every downstream token and cost number can be re-derived against the real config.
2. **Wire the synthetic de-duplication predicate into the build path.** It exists and is tested on 287,000 document ids, but is called from nothing that writes corpus data — so declared synthetic volume currently rests on about 28% as many distinct documents.
3. **Fix the load-balancing scope** (§11) — a trainer setting, before training starts.

Separately, and not a blocker for this corpus: the **missing end-of-sequence token** in the **SuperBPE** tokenizer, where an empty special-token list makes the validator's EOS check skip silently. **That affects two already-published corpora and does not apply here** — this corpus is dolma2, whose `eos_token_id` is 100257 and whose EOS check therefore runs. It is listed only so the defect is not forgotten; fixing it changes nothing about `final-dataset`.

1. **Drop the source that ships GSM8K in chain-of-thought format** — 0.51% of tokens (§10).
2. **Decide the label schema** (§11) — this one is unbackfillable.
3. **Measure the three blocking unknowns** (§12), and measure **in-region bandwidth** first — ten minutes, and it calibrates every duration estimate in the implementation plan.
4. **⚠ Freeze this mix completely, and only then ingest.** An earlier version of this line read "*then ingest, source by source, each as a separately approved job.*" **That is superseded, and following it destroys work:** shard ordinals are allocated across the whole plan in alphabetical order, so adding one source renames **98% of shards** and voids **882B tokens** of finished tokenization. Sources may still be *approved* and *run* one at a time — but **the plan must cover all of them before the first job runs.**

Every AWS job goes through the platform submission form and requires a human release. Nothing auto-publishes.